

Do Diverse Sequence Architectures Learn the Same Music? A CKA Analysis of Representational Convergence from Classical ML to State Space Models

Daniel Ju

The George Washington University

1. Introduction

1.1 Motivation

Music genre classification is a canonical sequence classification task in music information retrieval (MIR). A 30-second audio clip converted to a mel spectrogram at standard resolution yields a sequence of approximately 1,292 time frames:

$$T = \left\lfloor \frac{30 \times 22,050}{512} \right\rfloor + 1 = 1,292$$

(The exact frame count depends on STFT centering and padding; `center=True` may yield $\sim 1,296$ frames. We report the uncentered calculation and confirm tensor shapes empirically in preprocessing.)

A single audio frame contains 128 mel-frequency bins, placing genre classification in a regime where long-range temporal modeling is architecturally consequential. The evolution of sequence classification architectures reflects distinct strategies for compressing and representing long temporal structure:

- **Classical ML** (SVM, Random Forest, XGBoost) operates on hand-crafted summaries (MFCCs, chroma, spectral contrast) that map an audio clip to a fixed-length vector, encoding established DSP priors (Tzanetakis & Cook, 2002).
- **1D-CNNs** capture local spectro-temporal patterns through bounded receptive fields, relying on depth for longer-range aggregation (Liu et al., 2019; Zhang & Li, 2025).
- **LSTMs** recurrently update a hidden state that compresses sequence history into a fixed-dimensional representation (Tang et al., 2018; Feng et al., 2017).
- **Transformers** retain global access via self-attention, avoiding explicit compression but incurring quadratic complexity (Vaswani et al., 2017).
- **Mamba (selective SSM)** performs input-dependent state updates that selectively compress sequences with linear-time scaling (Gu & Dao, 2024; Dao & Gu, 2024).

Despite this diversity, no prior work evaluates the **full architectural spectrum** - classical feature-based models

through CNN, LSTM, Transformer, and notably in this paper, selective state space models - on a standard MIR benchmark under controlled from-scratch training with representational analysis. To date, only MoMamba (Liu et al., ICLR 2026 submission) compared a Mamba variant against CNN, LSTM, and Transformer baselines from scratch, but excluded classical ML baselines and included no representational analysis and was ultimately withdrawn. The MARBLE benchmark (Yuan et al., NeurIPS 2023 D&B) standardizes evaluation for music understanding tasks but emphasizes pretrained representations, leaving from-scratch architectural comparison underexplored.

Prior benchmarking work would confound architectural inductive bias with pretraining scale, dataset differences, and inconsistent evaluation protocols. By training all deep learning models **from scratch** on identical data under identical conditions, this study isolates architectural inductive bias as the independent variable. This is a deliberate departure from MARBLE’s pretraining-focused protocol as the paper asks a different and complementary question: *what does each architecture learn from the raw signal alone, absent pretrained knowledge?*

Beyond performance comparison, this paper asks a deeper question about **what architectures actually learn**, and whether they are truly discovering musical structure or exploiting statistical shortcuts. Each architecture encodes a different **inductive bias**: structural assumptions baked into the model before training on patterns in the data. For example, LSTM assumes sequential order where context accumulates over time, Transformer assumes all positions may be relevant to all others, and Mamba assumes sequence should be selectively compressed into a finite state. These assumptions are not “learned”; instead, are consequences of how each model is wired, determining what each architecture finds “easy” versus “hard” to learn from the same data.

If a Mamba model and a Transformer achieve similar accuracy on genre classification, are they encoding the same information about the music - or fundamentally different information? And how do their learned representations relate to the hand-crafted features (MFCCs, chroma, spectral contrast) that DSP engineers designed over decades to capture musically meaningful structure? Answering these questions moves beyond the “which model wins?” framing towards a

more fundamental understanding of *how* and *why* different architectures succeed or fail, building interpretability that informs not just model selection but architectural design.

We investigate these questions using Centered Kernel Alignment (CKA) (Kornblith et al., 2019), a representation similarity metric that can compare internal representations across architectures with different hidden dimensionalities. By decomposing CKA alignment into feature-group-specific scores (alignment with MFCCs, with chroma, with spectral contrast separately), we construct **interpretable alignment profiles** for each architecture to reveal not just *how similar* learned representations are to classical features, but *which specific aspects of musical structure* each architecture learns to encode.

1.2 Research Questions & Hypotheses

Our primary research questions for this project are scoped as:

- **RQ1 (Performance):** How do architectures spanning the full spectrum from classical ML to Mamba-2 compare on 16-class genre classification when trained from scratch under controlled conditions?
- **RQ2 (Representation):** Do architecturally distinct deep learning models converge toward a shared representational subspace - and does that subspace align with classical hand-crafted audio features?
- **RQ3 (Interpretability):** Can feature-group-specific CKA alignment profiles explain per-genre confusion patterns across architectures?

We frame our study around a falsifiable, hierarchical hypothesis structure:

H1 (Performance Baseline): Mamba-1 and Mamba-2 match or exceed vanilla Transformer on macro F1 (class-balanced F1 averaged across genres) and accuracy for 16-class genre classification when trained from scratch, while requiring **comparable or reduced** peak GPU memory and wall-clock training time at the standard $\sim 1,292$ -token sequence length. At longer sequences (Experiment 4, $\geq 5,000$ tokens), we predict Mamba 1-2 will require **strictly less** peak GPU memory, as Transformer self-attention scales quadratically ($O(N^2)$) while Mamba’s selective scan scales linearly ($O(N)$).

Why “comparable or reduced” at 1,292 tokens rather than “strictly less”: At short sequence lengths, quadratic attention cost remains modest, particularly with FlashAttention (Dao, 2023), which fuses attention into a memory-efficient kernel. Under these conditions, Transformer memory usage at $\sim 1,292$ tokens may approximate Mamba’s due to state-matrix overhead. The SSM scaling advantage emerges clearly only at longer sequences. We therefore use PyTorch’s `scaled_dot_product_attention`, which selects optimized attention backends (including FlashAttention-2 where available), ensuring a modern

Transformer baseline rather than a naive $O(N^2)$ -memory implementation.

Rationale: SSM theory predicts linear-time scaling with competitive accuracy (Gu & Dao, 2024), establishing this hypothesis as a necessary empirical baseline. Confirmation validates Mamba as a viable architecture for audio classification and verifies experimental fidelity prior to representational analysis. Falsification would indicate domain-dependent limitations of SSMs and challenge claims of universal sequence-model replacement.

H2 (Representational Convergence): All deep learning architectures will converge towards a shared representational subspace at the penultimate layer. Operationally, convergence is defined as: inter-architecture pairwise CKA approaches the within-architecture cross-seed CKA ceiling - i.e., a Mamba model’s representation is approximately as similar to a Transformer’s representation as two independently trained Transformers are to each other. CKA significantly exceeding the untrained-network baseline is a necessary but insufficient condition - it proves learning occurred, not convergence. Furthermore, each architecture’s CKA alignment with the classical hand-crafted feature space will correlate positively with its classification performance - the architectures that learn representations most similar to classical features will classify most accurately.

Rationale: Genre classification may rely on low-dimensional discriminative structure already captured by MIR feature engineering (Tzanetakis & Cook, 2002; Mishra & Akhtar, 2025). MFCCs encode timbral texture, chroma vectors harmonic content, and spectral contrast tonal-noise balance - perceptually grounded attributes designed to capture musically salient structure. Convergence toward these features would imply deep models approximate an established discriminative signal. Related CKA findings in NLP (Hoang et al., 2026) show shared representational structure across Transformer and Mamba models despite differing frameworks, suggesting convergence may generalize across sequence domains. MIR evidence that hand-crafted features can rival learned representations on some tasks (Plachouras et al., 2025) further supports this possibility.

Implication if confirmed: Classical feature engineering effectively solved the representation problem for genre classification, positioning deep learning gains primarily in efficiency.

Implication if falsified: Architectures discover distinct musical abstractions beyond classical DSP. A plausible alternative - classical-feature alignment peaking in early layers and declining deeper - has been observed in speech sequence models (Ollerenshaw et al., 2022); the intermediate-layer CKA analysis in §4.3.2 explicitly tests this pattern.

H2a (DSP Convergence): Mamba’s penultimate-layer representations exhibit higher CKA alignment with classical hand-crafted features than Transformer’s representations, despite both achieving comparable classification ac-

curacy.

Rationale: Mamba’s selective state space computation is modeled as linear recurrence with input-dependent gating, structurally related to the infinite impulse response filtering underlying MFCC computation. MFCC pipelines consist of cascaded linear filtering operations (mel filterbank followed by DCT), whereas Mamba generalizes this class via adaptive filter coefficients. This structural proximity may bias Mamba representations toward classical DSP features more than Transformer attention mechanisms.

Implication if confirmed: Mamba’s effectiveness on audio stems from alignment with established signal-processing priors - architectural rediscovery of DSP principles.

Implication if falsified: The intuitive SSM-filter analogy is weakened, implying selective mechanisms capture qualitatively different structure. Notably, evidence of weaker local feature extraction in pure Mamba variants (Liu et al., 2026) suggests potential divergence from window-based spectral statistics underlying classical features.

H3 (Interpretable Error Patterns): Per-architecture confusion patterns are explained by feature-group-specific CKA alignment profiles. Specifically: architectures with low CKA alignment to chroma features disproportionately confuse genres that differ primarily in harmonic content, and architectures with low alignment to MFCCs disproportionately confuse genres differing in timbral texture.

Rationale: This hypothesis links inductive bias, representation structure, and classification errors within a unified interpretive framework grounded in musicological distinctions. For example, chroma features capture pitch-class energy distributions relevant to harmonic vocabulary, while MFCCs encode timbral texture associated with instrumentation and production style.

Implication if confirmed: Prediction of genre confusions from representational properties becomes possible, converting confusion matrices into interpretable artifacts of learned structure.

Implication if falsified: A more complex relationship between learned representations and errors is implied, motivating nonlinear probing approaches.

2. Related Work

This section surveys prior work in genre classification, structured state space models for audio, representation similarity analysis, and benchmarking standards.

2.1 Music Genre Classification

Classical feature-engineered approaches are defined primarily by Tzanetakis & Cook (2002), with their foundational work establishing MFCCs, spectral centroid, spectral rolloff, and zero-crossing rate as genre-discriminative features on GTZAN (1,000 tracks, 10 genres). These features compose the standard feature set that persists in MIR. Mishra

& Akhtar (2025)’s comparative study of SVM + ensemble methods vs. CNN on GTZAN demonstrates classical feature-engineered models outperform CNN in data-limited settings, directly motivating our inclusion of classical baselines and raising the question central to H2: does deep learning learn anything beyond what hand-crafted features already provide? Chatterjee et al. (2024) evaluate Logistic Regression, KNN, Random Forest, and ANN on GTZAN using pattern-recognition features. Their work shows ANN outperforms classical baselines but emphasizes the continued relevance of feature engineering.

Work in CNN-based genre classification is composed of Liu et al. (2019), who utilized a Bottom-up Broadcast Neural Network CNN architecture for spectrogram-based genre classification on GTZAN and Ballroom, representative of a CNN-dominant MIR baseline. Zhang & Li (2025) approached classification with a Parallel CNN ensemble using MFCC, STFT, and DWT features, demonstrating continued CNN architecture innovation within MIR. Luo (2023) did a direct CNN vs. LSTM comparison on GTZAN and FMA, and found CNN outperformed LSTM across both datasets. These works establish the CNN > LSTM ordering on this specific task, which our study can either confirm or challenge under different conditions.

The only comparable work regarding modern deep-architecture comparisons in genre classification is Chu Ba et al. (2025), who evaluated CNN, LSTM, GRU, and capsule networks with augmentation on GTZAN. This work notes high accuracy across architectures but does not include Transformer or SSM baselines.

In conclusion, most prior genre classification work compares 2-3 architectures (typically CNN vs. LSTM or CNN vs. ANN) on GTZAN (1,000 tracks, 10 genres) without controlling for training conditions or evaluating subspaces for interpretability. No published work spans the full classical-to-SSM spectrum on FMA Medium (25,000 tracks, 16 genres) under the MARBLE evaluation protocol. Our study fills this gap with controlled from-scratch training, five-seed statistical testing, and representational analysis.

2.2 Structured State Space Models for Audio

We view SSM foundational work in audio classification tasks as built on Gu, Goel & Ré (2022)’s S4 model built on SSM, which introduced the HiPPO framework for long-range dependency modeling via continuous-time state transitions. S4 serves as historical context: it is the direct precursor to Mamba and the theoretical foundation (HiPPO initialization, structured state matrices) on which Mamba builds. S4 is not included in our experiments because Mamba supersedes it with the selective mechanism; S4’s fixed (non-input-dependent) state transitions make it a less interesting architectural comparison point than the selective variants. Gu & Dao (2024) pioneered Mamba - selective SSMs with input-dependent gating of the B, C, and Δ parameters - achieving

linear-time scaling with content-aware processing. Dao & Gu (2024) later built Mamba-2, characterized by structured state space duality, and demonstrating SSMs and Transformers are more closely related than previously understood.

To date, Mamba in music genre-classification is non-existent. MoMamba (Liu et al., ICLR 2026 submission, withdrawn) is the most directly competing prior work, utilizing a lightweight Mamba-based music classification model evaluated on several MIR benchmarks including genre classification, trained from scratch without pretraining and compared against CNN and Transformer baselines.

Key differentiations: (1) MoMamba excludes classical ML baselines (SVM, RF, XGBoost); our study includes these as both performance baselines and the reference point for CKA alignment analysis. (2) MoMamba includes no representational analysis (CKA or otherwise); representational convergence is our primary contribution. (3) MoMamba is withdrawn and unpublished.

Other work involving audio includes Yadav & Tan (2024), who proposed self-attention-free Mamba for general audio classification using self-supervised pretraining. This work is pretraining-based; our from-scratch approach answers a complementary question about raw architectural inductive bias. Shams et al. (2024) evaluated Mamba against SSAST on downstream audio classification - this is a self-supervised paradigm, and not directly comparable to our from-scratch setup. Mu et al. (2024) built SELD-Mamba, a selective state-space model for sound event localization and detection with source distance estimation using bidirectional Mamba blocks in an Event-Independent Network framework. This work is task-adjacent but further demonstrates Mamba viability for audio sequence tasks. Bairwa et al. (2025) provides another Mamba-vs-Transformer-vs-CNN comparison via StructMamba for music genre but omits classical ML baselines, LSTM, and representational analysis - reinforcing the gap our work fills. Finally, Fratticioli (2025) grounds our work through MambaTransfer - a raw waveform Mamba for timbre transfer, demonstrating SSM applicability to raw music signals.

Existing Mamba-audio work focuses on either (a) pretraining-based evaluation or (b) proposing a new Mamba variant. None provides the cross-architecture representational analysis that constitutes our primary contribution. Our work is methodologically complementary to MoMamba: where MoMamba asks “does a music-specific Mamba variant improve performance?” (comparing against CNN, LSTM, and Transformer baselines), we ask “what do architectures learn, and do they converge toward the same representation?” - while additionally bridging the gap to classical feature-engineered baselines that MoMamba omits.

2.3 Representation Similarity Analysis

Our CKA methodology is informed by Kornblith et al. (2019), who introduced CKA for comparing neural net-

work representations. Their work shows networks trained from different initializations learn highly similar features, while prior methods (CCA, SVCCA) failed to detect this, serving as our methodological foundation for our representational analysis. Raghu et al. (2022) applied CKA to compare Vision Transformers vs. CNNs, and found striking architectural differences: ViTs have more uniform representations across layers; CNNs show hierarchical feature progression. Though this work operates in a different classification task, it is directly analogous to our cross-architecture comparison - we extend Raghu et al.’s methodology from vision (CNN vs. ViT) to audio (CNN vs. LSTM vs. Transformer vs. Mamba).

There are some limitations with CKA - Nguyen et al. (2021) demonstrated CKA sensitivity to outliers and certain invertible linear transformations. We mitigate this issue by reporting both linear CKA and RBF CKA, and by supplementing CKA with classification probe accuracy as a convergent validity check.

CKA has not currently been used specifically in genre classification tasks, but methodological precedents exist in audio and speech. Ollerenshaw et al. (2022) applied CCA and CKA to analyze internal representations of CNN, LSTM, and Transformer architectures during end-to-end ASR training, and found CNNs exhibit progressive hierarchical similarity, LSTMs show bottom-up convergence, and Transformers show irregular layer-correlation patterns. Though not genre-classification, this work establishes CKA can be a viable cross-architecture comparison tool in audio. Pasad et al. (2021)’s comparative study of self-supervised speech representations using linear CKA and SVCCA across RNN, Transformer, and CNN architectures further demonstrates CKA methodology for comparing representation geometry across architectural families in the speech domain. Their later work, Pasad et al. (2023), used linear CKA to compare classical acoustic features (e.g., MFCC-based pipelines) with neural representations across layers of speech models for phone and speaker classification. This directly informs our methodology - CKA between hand-crafted features and learned representations - though in speech rather than music.

CKA has also been applied across SSM / Mamba architectures, setting a cross-domain precedent. Lou et al. (2025) applied CKA in SparX, a sparse cross-layer connection mechanism for hierarchical Vision Mamba and Transformer networks, extending Raghu et al.’s CNN-vs-ViT CKA analysis to include SSMs, though in vision, not audio. Hoang et al. (2026) conducted a comparative analysis of contextual representation flow in state-space and Transformer architectures via CKA, and found that despite fundamentally different mathematical foundations, the architectures share significant representational structure, with a high-dimensional abstraction phase in middle layers. This informs H2 as the strongest existing evidence for representational convergence

between Transformer and Mamba, albeit in NLP.

CKA has been applied extensively in computer vision to compare CNN vs. Transformer representations (Raghu et al., 2022), and the methodology has been established in audio/speech for cross-architecture comparison (Ollerenshaw et al., 2022; Pasad et al., 2021) and for comparing learned representations against classical acoustic features (Pasad et al., 2023). Recent work has extended CKA analysis to include SSMs in vision (Lou et al., 2025) and NLP (Hoang et al., 2026), finding significant representational convergence between Transformer and Mamba models. However, no published work applies CKA to the full classical-ML-to-SSM architectural spectrum on an MIR genre benchmark, nor measures alignment between learned deep representations and named, interpretable classical feature groups (MFCCs, chroma, spectral contrast) to explain classification behavior. The combination of SSM-inclusive cross-architecture CKA with feature-group decomposition for interpretability in the music domain defines this paper’s novel contributions.

2.4 Benchmarking Standards

We rely on MARBLE (Yuan et al., NeurIPS 2023 D&B) - a standardized benchmark for music representation learning. MARBLE defines a unified protocol across 18 tasks on 12 public datasets, including FMA for genre classification, and specifies train/val/test splits, evaluation metrics (macro F1, accuracy). We adopt MARBLE’s evaluation protocol (splits, metrics) but deliberately depart from its pretraining paradigm to isolate architectural inductive bias from pre-training scale - a different and complementary experimental question. FMA (Defferrard et al., 2017) refers to the Free Music Archive dataset, providing audio and pre-computed features for 106,574 tracks across 161 genres.

3. Dataset

This section describes the FMA Medium dataset, preprocessing pipeline, feature groups for CKA analysis, and data splits.

3.1 Dataset Overview

We use FMA Medium as evaluated under the MARBLE benchmark protocol (Yuan et al., NeurIPS 2023 D&B).

Genre distribution: We expect FMA Medium to exhibit moderate class imbalance (exact per-genre counts to be reported). Weighted cross-entropy loss addresses this during training (§4.2).

Justification for FMA Medium over alternatives:

- **vs. GTZAN** (Tzanetakis & Cook, 2002; 1,000 tracks, 10 genres): Too small for reliable deep learning training, known label errors, and saturation effects from widespread use. FMA Medium is $25\times$ larger with 60% more genre classes.

Table 1: Dataset overview: FMA Medium under the MARBLE benchmark protocol.

Property	Value	Source
Dataset	Free Music Archive (FMA) Medium subset	Defferrard et al. (2017)
Benchmark protocol	MARBLE	Yuan et al. (NeurIPS 2023 D&B)
Tracks	25,000	
Genres	16 (single-label per track)	
Audio format	30-second clips, 22,050 Hz, mono MP3	
Pre-computed features	MFCCs (20 coefficients), chroma (12 bins), spectral contrast (7 bands), + additional	Defferrard et al. (2017); extracted via librosa (McFee et al., 2015)

- **vs. FMA Large** (106,574 tracks): Compute-prohibitive for our full architectural setup (8 models $\times$ 5 seeds).
- **vs. FMA Small** (8,000 tracks, 8 genres): Viable, but insufficient scale to stress-test deep architectures and too few classes for meaningful confusion matrix analysis in H3 compared to FMA Medium.

3.2 Data Preprocessing & Cleaning Pipeline

3.2.1 Audio Preprocessing (Deep Learning Models)

All deep learning models receive identical input derived from the following pipeline:

Step 1. Audio loading and sample rate standardization. Each track is loaded from the FMA Medium MP3 files using librosa (McFee et al., 2015), which decodes compressed audio to a floating-point waveform. All tracks are resampled to 22,050 Hz (FMA’s native sample rate) to ensure every track enters the pipeline at a consistent sample rate regardless of source encoding.

Step 2. Mono conversion. Stereo tracks are converted to mono via channel averaging. Genre classification does not depend on stereo spatial imaging; mono conversion reduces dimensionality without task-relevant information loss.

Step 3. Mel spectrogram extraction. This step converts the raw waveform into a time-frequency representation - effectively “slicing” the 30-second audio clip into overlapping analysis windows and computing the frequency content of each window. The result is a 2D image-like representation where one axis is time (frames) and the other is frequency (mel bins).

All parameters follow standard MIR conventions (McFee et al., 2015; Choi et al., 2017; Yuan et al., 2023). The `n_fft` of 2,048 yields a 93 ms analysis window with minimum

Table 2: Mel spectrogram extraction parameters.

Parameter	Value
Sample rate	22,050 Hz
n_fft	2,048 samples
hop_length	512 samples
n_mels	128
fmin	0 Hz
fmax	11,025 Hz (Nyquist)
Window function	Hann
Mono conversion	Channel averaging
Clip handling	Truncate/zero-pad to 30 s

resolvable frequency of 10.8 Hz, sufficient for the lowest musical fundamentals. The `hop_length` of 512 yields 43 frames/second with 75% overlap, providing adequate temporal resolution for genre-level rhythmic patterns. Mel-scale frequency bins (128) balance resolution with cost. We retain `fmin` = 0 Hz for maximum comparability with default librosa pipelines and use the Nyquist frequency (11,025 Hz) as the upper edge.

Resulting sequence dimensions:

$$\text{Time frames} = \left\lfloor \frac{30 \times 22,050}{512} \right\rfloor + 1 = 1,293 \text{ frames}$$

Each deep learning model receives input of shape (**128 mel bins** $\times$ **1,292 time frames**). The 128-dimensional mel vector at each time frame is treated as a token, yielding a sequence of $\sim$ **1,292 tokens** per 30-second clip. By design, this is the identical input format fed to all deep learning architectures - ensuring that any differences in learned representations or classification performance are attributable to architectural inductive bias, not to differences in input preprocessing.

Step 4. Log-amplitude normalization. Given raw mel spectrogram S (power scale), we compute $S_{\log} = \log_{10}(1 + S)$, then per-track z-score: $S_{\text{norm}} = (S_{\log} - \mu_{\text{track}})/\sigma_{\text{track}}$, where μ and σ are the mean and standard deviation of all values in that track’s spectrogram. We use $\log(1 + S)$ rather than librosa’s `power_to_db` (which uses a fixed or peak reference) because per-track z-scoring already removes absolute scale - the reference value in dB conversion would be normalized away. Log-amplitude maps perceptual loudness more linearly than raw amplitude; per-track normalization removes recording-level volume differences while preserving within-track dynamics.

We choose log-amplitude as raw magnitude spectrograms have extreme dynamic range (quiet and loud regions differ by orders of magnitude), making gradient-based optimization unstable. Power-law compression ($S^{0.3}$) is an alternative used in some speech systems (PCEN), but $\log(1 + S)$ is the most widely adopted normalization in MIR (Choi et al., 2017; Yuan et al., 2023), and using a non-standard normal-

Table 3: SpecAugment parameters (applied during training only).

Parameter	Value
Number of frequency masks	2
Max frequency mask width	20 mel bins
Number of time masks	2
Max time mask width	50 frames

ization would reduce comparability with prior work. Note that this normalization operates on each track independently and does *not* address class imbalance across genres - that is handled by weighted cross-entropy loss (§4.2).

Step 5. SpecAugment (Park et al., 2019).

SpecAugment is applied during training only with frequency masks up to 20 mel bins masked per sample and with time masks, up to 50 frames masked per sample.

SpecAugment is the standard data augmentation for spectrogram-based audio models (Park et al., 2019). It operates directly on the spectrogram, introducing no audio-domain artifacts. No other augmentation (pitch shift, time stretch, noise injection) is applied, maintaining clean experimental control over the single variable of interest: architectural inductive bias.

We rejected the following preprocessing alternatives:

- **Raw waveform input** requires significantly deeper models and more compute to learn equivalent spectral features. Our research question concerns representational convergence at the spectrogram level; raw waveform experiments are deferred to future work.
- **CQT (Constant-Q Transform)** provides logarithmic frequency resolution (better for pitch tracking), but mel spectrograms are the standard preprocessing in MARBLE and the majority of MIR literature. Using CQT would confound preprocessing choice with architecture comparison.
- **Multi-resolution spectrograms** would complicate our “identical input across architectures” requirement.

3.2.2 Feature Preprocessing (Classical ML Models)

Classical ML models (SVM, Random Forest, XGBoost) use FMA’s pre-computed feature CSVs (Defferrard et al., 2017), extracted via librosa.

- 20 MFCCs (mean and variance per track $\rightarrow$ 40 features)
- 12 chroma features (mean and variance $\rightarrow$ 24 features)
- 7 spectral contrast bands (mean and variance $\rightarrow$ 14 features)
- Additional: spectral centroid, bandwidth, rolloff, zero-crossing rate, tonnetz (mean and variance)

- Total: ~ 518 features per track (exact count confirmed from FMA metadata)

Standard scaling (zero mean, unit variance) fit on the training set and is applied to validation and test sets; no additional feature engineering is applied. We use pre-computed features rather than extracting from mel spectrograms, as the pre-computed features represent the standard feature set established by MIR literature (Tzanetakis & Cook, 2002). Using them (a) ensures fair comparison with prior classical ML results, (b) follows established practice, and (c) critically, provides the **named, interpretable feature groups** (MFCCs, chroma, spectral contrast) required for the CKA alignment analysis in H2/H3.

3.3 Feature Groups for CKA Analysis

The CKA representational alignment analysis (H2, H2a, H3) requires decomposing the classical feature space into interpretable groups. This decomposition is not arbitrary - each group corresponds to a distinct, well-understood dimension of musical structure identified and validated by MIR research. MFCCs were established as timbral descriptors by Tzanetakis & Cook (2002) and remain the most widely used audio features in MIR (Logan, 2000). Chroma features were formalized by Bartsch & Wakefield (2005) as representations of harmonic content. Spectral contrast was introduced by Jiang et al. (2002) specifically to discriminate music types by measuring peak-to-valley differences across frequency sub-bands.

Feature group independence: The interpretability of H3 depends on these feature groups capturing *distinct* musical dimensions. If MFCCs and spectral contrast are highly correlated across the dataset (both derive from spectral envelope properties), then attributing a confusion pattern to “low chroma alignment” vs. “low MFCC alignment” becomes ambiguous. We address this in §4.3.3 with a preliminary collinearity analysis.

3.4 Data Splits (Train / Validation / Test)

We adopt MARBLE-standardized splits for FMA Medium (Yuan et al., 2023). These splits are **artist-stratified**: no artist appears in more than one split. This is critical because without artist stratification, a model could “cheat” by recognizing an artist’s recording style, production choices, or vocal timbre rather than learning genre-level features - a well-documented source of data leakage in MIR evaluation (Flexer & Schnitzer, 2010; Defferrard et al., 2017). If Artist X appears in both train and test, the model may learn “this sounds like Artist X $\rightarrow$ genre Y” rather than “this harmonic/timbral pattern $\rightarrow$ genre Y.” Artist stratification forces the model to generalize to *unseen artists* within each genre, testing genuine genre-level representation learning.

Exact counts to be confirmed from MARBLE split specification. No test-set access occurs during model develop-

Table 4: Feature groups used in CKA representational alignment analysis.

Feature Group	Dim.	Musicological Interpretation
MFCCs	40 (20 coeff. $\times$ mean/var)	Timbral texture - the “color” or “quality” of sound. Encodes instrumentation, vocal characteristics, production style. Derived from the spectral analysis of the mel-scale power spectrum.
Chroma	24 (12 bins $\times$ mean/var)	Harmonic/pitch content - the distribution of energy across the 12 pitch classes (C, C#, ..., B). Captures key signature, harmonic complexity, chord vocabulary.
Spectral contrast	14 (7 bands $\times$ mean/var)	Peak-to-valley spectral energy distribution - distinguishes tonal (peaked) from noisy (flat) spectral content across frequency bands. Captures genre-specific frequency balance.

Table 5: MARBLE-standardized artist-stratified data splits for FMA Medium.

Split	Approximate Size	Purpose
Train	$\sim 19,922$ tracks	Model training
Validation	$\sim 2,573$ tracks	Hyperparameter tuning, early stopping criterion
Test	$\sim 2,505$ tracks	Final evaluation; all reported metrics computed here

ment or hyperparameter tuning.

4. Methods

This section details the model architectures, training procedure, experimental setup, and evaluation protocol.

4.1 Model Architectures

The architecture lineup is chosen to span the full history of approaches to sequential classification, with each model representing a distinct inductive bias for sequence compression. The progression from classical ML through Mamba traces the conceptual evolution from “compress first, classify second” (classical ML on hand-crafted features) to “learn to compress and classify jointly” (deep learning), and within deep learning, from local (CNN) to recurrent (LSTM) to global (Transformer) to selective (Mamba) compression strategies.

4.1.1 Classical ML Baselines

All classical models operate on pre-computed feature vectors (§3.2.2), not on mel spectrograms.

SVM (Support Vector Machine): RBF kernel. Hyperparameters (C , γ) tuned via grid search on the validation set. SVM with RBF kernel is a strong baseline for classification in moderate-dimensional feature spaces. It represents kernel-based discrimination without learned feature extraction.

Random Forest: 500 trees; max depth tuned on the validation set. Represented through an ensemble of decision trees with feature subsampling, capturing non-linear feature interactions through recursive partitioning. Random Forest provides a complementary non-parametric baseline to SVM.

XGBoost: Gradient-boosted trees, with learning rate, max depth, and number of estimators tuned via grid search on the validation set. XGBoost represents state-of-the-art classical ML for tabular/structured data, and serves as the strongest expected classical baseline. If XGBoost on hand-crafted features approaches deep learning performance, it strengthens the case that genre-discriminative information is largely captured by classical features (supporting H2).

Motivation for classical ML baselines: Classical ML baselines are included for three reasons: (a) Mishra & Akhtar (2025) showed classical models can outperform deep learning in data-limited MIR settings - this must be tested at FMA Medium’s scale. (b) The classical feature vectors serve as the **reference representation** for CKA alignment analysis; without them, H2 and H2a are untestable. (c) Classical ML accuracy on hand-crafted features establishes the performance ceiling of what those features alone can achieve, providing the interpretive baseline against which deep learning’s learned representations are compared.

4.1.2 Deep Learning Models

All deep learning models receive identical input: mel spectrogram of shape (128 mel bins $\times$ 1,292 time frames). Each mel vector at each time step is treated as a 128-dimensional token, yielding a sequence of 1,292 tokens. Each model produces a 16-dimensional logit vector for genre classification.

1D-CNN:

Architecture: 4 convolutional blocks (Conv1D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool), global average pooling, linear classification head. Kernel sizes: 7 (first layer), 5 (subsequent layers) with channels: 64 $\rightarrow$ 128 $\rightarrow$ 256 $\rightarrow$ 512.

Inductive bias: Local pattern detection with bounded receptive field. Each layer captures patterns within a fixed temporal window; long-range dependencies require stacking multiple layers. The effective receptive field after 4 blocks with pooling covers a fraction of the full 1,292-frame sequence.

CNNs are the dominant architecture in pre-Transformer MIR (Liu et al., 2019; Zhang & Li, 2025; Luo, 2023). Including CNN establishes the “local feature detection”

baseline against which global (Transformer) and selective (Mamba) approaches are compared. For CKA analysis, the CNN’s alignment profile tests whether local receptive fields converge toward the same features as global attention.

The 1D-CNN baseline is intentionally shallow with small kernel sizes, representing the local-receptive-field inductive bias in its purest form. We do not use dilated convolutions, deep residual networks (e.g., ResNet), or Temporal Convolutional Networks (TCNs), as these architectural modifications approximate global context and would reduce the architectural contrast central to our analysis. The effective receptive field of this 4-block CNN (accounting for kernel sizes 7, 5, 5, 5 and max-pooling with stride 2 at each block) covers approximately $7 + 3 \times (5 - 1) \times 2^{0-3} \approx 100$ -200 frames (~ 2 -5 seconds), a small fraction of the full 1,292-frame sequence. This bounded receptive field is the defining characteristic of the CNN inductive bias under study.

LSTM (Bidirectional):

Architecture: 2-layer bidirectional LSTM, hidden dimension 256, attention-weighted temporal pooling, linear classification head. The LSTM serves as the **canonical gated-recurrent baseline representing the pre-attention paradigm**; it does not serve as a competitive modern architecture. While largely superseded by Transformers and SSMs for sequence modeling, LSTM’s inclusion is necessary to complete the architectural progression (local convolution $\rightarrow$ gated recurrence $\rightarrow$ global attention $\rightarrow$ selective state spaces) and to isolate which properties of modern architectures drive representational differences.

Why bidirectional: Genre-discriminative information may appear at any position in the clip (not just the end). Unidirectional LSTM would bias toward later time steps.

Why attention pooling instead of the final hidden state: The final hidden state discards temporal position information. Attention-weighted pooling allows the model to weight which time steps are most genre-informative; a learned temporal focus mechanism.

Inductive bias: Gated recurrent compression. The hidden state is updated at each time step through forget/input/output gates, creating a compressed running summary of the sequence. Unlike Transformer, LSTM cannot attend to arbitrary positions; unlike Mamba, its gating parameters are not input-dependent in the same structured way.

Bidirectional LSTM completes the recurrent $\rightarrow$ attention $\rightarrow$ selective SSM progression. Without LSTM, we cannot distinguish whether Mamba’s advantage (if any) comes from being a *better recurrent model* or from its *selective mechanism* specifically. LSTM also provides the baseline for Luo (2023)’s finding that CNN outperforms LSTM on genre; we test whether this holds under our controlled conditions.

Vanilla Transformer:

Architecture: 4-layer Transformer encoder, **sinusoidal positional encodings** (Vaswani et al., 2017), 4 attention heads, model dimension 256, FFN dimension 1024, [CLS]

token pooling, linear classification head. Sinusoidal positional encodings are the original positional encoding from Vaswani et al. (2017), computed from a fixed mathematical formula rather than learned parameters. This choice is motivated by two considerations: (1) sinusoidal encodings generalize to sequence lengths not seen during training, which is required for Experiment 4’s sequence length scaling - learned embeddings would have no weights for positions beyond the training length and would require retraining the Transformer from scratch at each new sequence length; (2) sinusoidal encodings maintain the “vanilla” Transformer framing - alternatives like Rotary Position Embeddings (RoPE; Su et al., 2024) also generalize to unseen lengths but are a post-2023 innovation that would complicate our “original architecture” framing. The performance difference between sinusoidal and learned embeddings at our standard training length ($\sim 1,292$ tokens) is typically negligible for classification tasks.

Vanilla, instead of pre-trained and/or hybrid approaches, were selected as it isolates self-attention’s inductive bias from pretraining knowledge. A pretrained Transformer (e.g., AST, HuBERT) would confound architectural bias with pre-training data. A hybrid (e.g., Conformer) would confound attention with convolution. This has the caveat where training a Transformer from scratch on $\sim 25,000$ tracks places it in a data regime where its lack of inductive bias may be a disadvantage relative to architectures with stronger structural priors (CNN’s locality, LSTM’s gated recurrence, Mamba’s selectivity). Transformers are known to require larger datasets to properly structure the attention mechanism when trained without pretraining. This is a deliberate experimental choice that tests raw inductive bias, not practical deployment performance - if Transformer underperforms, the data regime is a plausible contributing factor alongside architectural differences.

Inductive bias: Full-sequence global attention. Every position attends to every other position; no explicit compression occurs until the classification head. The model retains complete access to the raw sequence but must learn to extract discriminative patterns from quadratically many pairwise interactions.

Transformer is the dominant paradigm in modern sequence modeling. It is the direct comparison point for Mamba - both process the full sequence, but Transformer via attention and Mamba via selective state transitions. For CKA analysis, the Transformer’s alignment profile directly tests H2a: does global attention converge toward the same features as Mamba’s selective recurrence?

Mamba-1:

Architecture: 4-layer Mamba blocks (selective SSM + gated MLP), state dimension 16, model dimension 256, expand factor 2, convolution width 4, mean temporal pooling, linear classification head.

Inductive bias: Input-dependent selective state compres-

sion. The B , C matrices and step size Δ are functions of the input at each position (Gu & Dao, 2024), enabling the model to dynamically decide which parts of the sequence to propagate into the state and which to suppress. This is structurally related to IIR digital filtering with learned, input-dependent filter coefficients.

Mamba-1 is the foundational selective SSM architecture. Its linear-time scaling and content-aware selectivity represent a fundamentally different approach to sequence compression than both attention (Transformer) and fixed gating (LSTM).

Mamba-2:

Architecture: 4-layer Mamba-2 blocks (structured state space duality), state dimension 64, model dimension 256, mean temporal pooling, linear classification head. Mamba-2 reformulates the selective SSM of Mamba-1 via state space duality (Dao & Gu, 2024), using a scalar-times-identity structure for the state matrix A . This reveals a formal connection between SSMs and attention, and enables more efficient hardware utilization while maintaining selectivity.

The inclusion of Mamba-2 tests whether the architectural refinements in Mamba-2 yield measurably different representations (not just different performance) compared to Mamba-1. If Mamba-1 and Mamba-2 have near-identical CKA profiles despite different implementations, this suggests the *selective SSM inductive bias* matters more than specific architectural details.

Architecture sizing. All deep learning models are sized to have approximately comparable parameter counts (~ 2 -5M parameters; exact counts to be reported per model). This prevents confounding architectural inductive bias with model capacity. Although exact parameter matching is impossible due to different parameter-efficiency profiles across architectures, we report parameter counts and discuss implications.

Rejected architectures:

- **GRU:** Functionally similar to LSTM with fewer gates. Including both adds compute cost without a meaningfully distinct inductive bias.
- **Conformer / hybrid models:** Combines attention with convolution — would complicate the clean “one architecture = one inductive bias” framing; hybridized models are avoided in this experiment for this reason.
- **Pretrained models (AST, HuBERT, MERT):** Would confound pretraining data and scale with architecture; explicitly out of scope by design.

4.2 Training Procedure & Hyperparameters

Training protocol (all deep learning models):

AdamW is standard for Transformer/SSM training with decoupled weight decay. Learning rate and dropout are the only per-architecture hyperparameters, selected on valida-

Table 6: Training protocol for all deep learning models.

Parameter	Value
Optimizer	AdamW (Loshchilov & Hutter, 2019)
Learning rate	Tuned per arch.: {1e-4, 5e-4, 1e-3, 2e-3}
LR schedule	Cosine annealing, linear warm-up (5 epochs)
Batch size	32
Max epochs	100 (early stopping, patience = 10)
Loss function	Weighted cross-entropy
Seeds	5 per architecture
Dropout	Tuned per arch.: {0.0, 0.1, 0.2, 0.3}
Weight init.	PyTorch defaults; Mamba: HiPPO-based

tion macro F1. Batch size 32 is standard for audio classification and fits within A100 40 GB VRAM for all architectures. Weighted cross-entropy uses weights inversely proportional to class frequency, addressing FMA class imbalance. Five seeds enable mean $\pm$ std reporting and statistical significance testing. HiPPO-based A matrix initialization (Gu & Dao, 2024) is critical for SSM convergence.

Hyperparameter search scope: Learning rate and dropout are tuned per architecture on the validation set. All other hyperparameters are fixed across architectures to minimize confounds. Full hyperparameter configurations will be reported in the final paper and released as configuration files.

Loss function rationale: We use weighted cross-entropy rather than alternatives for three reasons: (1) FMA Medium exhibits moderate class imbalance, and inverse-frequency weighting ensures underrepresented genres contribute proportionally to the gradient signal. (2) Cross-entropy is the standard loss for multi-class classification and the default in MARBLE evaluations, maximizing comparability with prior work. (3) Alternatives considered: *focal loss* (Lin et al., 2018) down-weights easy examples and could help with class imbalance, but introduces an additional hyperparameter (γ) that would need per-architecture tuning, adding confounds. *Label smoothing* was considered but rejected for similar reasons - it changes the effective target distribution, and different architectures may respond differently to smoothed targets, complicating the “identical training conditions” requirement.

Implementation details:

Parallelization. Each (architecture $\times$ seed) combination is an independent training run, making the 25 deep learning experiments (5 architectures $\times$ 5 seeds) trivially parallel. On a single A100 40 GB, runs must be executed sequentially, yielding an estimated $\sim$ 25-30 GPU-hours total; on a multi-GPU cloud platform with concurrent instance provisioning, wall-clock time reduces to the duration of the single longest run ($\sim$ 5 hours). Classical ML training (SVM, RF, XGBoost) runs on CPU and completes in minutes. CKA computation is a post-hoc matrix operation on extracted representations and completes in minutes regardless of hardware. The A100’s 40

Table 7: Implementation details: tools and platforms.

Component	Tool / Platform
Compute	A100 40 GB VRAM; providers TBD
Deep learning framework	PyTorch (≥ 2.1) with CUDA
Mamba-1 implementation	mamba-ssm (official package; Gu & Dao, 2024)
Mamba-2 implementation	mamba-ssm v2 (official; Dao & Gu, 2024)
Transformer implementation	Custom PyTorch <code>nn.TransformerEncoder</code>
CNN implementation	Custom PyTorch <code>nn.Conv1d</code> blocks
LSTM implementation	PyTorch <code>nn.LSTM</code> with custom attention pooling
Classical ML	<code>scikit-learn</code> (SVM via <code>SVC</code> , Random Forest via <code>RandomForestClassifier</code>); <code>xgboost</code> (<code>XGBClassifier</code>)
Audio preprocessing	<code>librosa</code> ≥ 0.10 (McFee et al., 2015)
CKA computation	<code>torch-cka</code> library or custom implementation following Kornblith et al. (2019), executed on GPU
Experiment tracking	Weights & Biases (<code>wandb</code>)
Statistical testing	<code>scipy.stats</code> (paired t-test, Wilcoxon, Spearman; methodology per Demšar, 2006)
Distance computation (Exp 3)	<code>scipy.spatial.distance</code> (Euclidean genre-centroid distances)

GB VRAM comfortably accommodates batch size 32 at the standard $\sim$ 1,292-token sequence length for all architectures; for Experiment 4’s longest sequences ($\sim$ 12,919 tokens), the A100 provides sufficient headroom without requiring gradient accumulation or model parallelism.

All code, configuration files, and SLURM job scripts for reproducing results will be released upon publication. The dataset (FMA Medium) is publicly available.

4.3 Experimental Setup

4.3.1 Experiment 1: Performance Benchmarking (H1)

All 8 models are trained on the FMA Medium training set. For each model, report on the held-out test set: - Macro F1 (primary metric, per MARBLE protocol) - Accuracy (secondary metric, for comparability with prior work) - Per-class precision and recall (for H3 error analysis) - 16×16 confusion matrix

For deep learning models: report mean $\pm$ std across 5 random seeds. Report wall-clock training time (to convergence/early stopping), peak GPU memory, and inference throughput (tracks/second).

Statistical Significance: Following established methodology for comparing classifiers (Demšar, 2006), for each

pair of deep learning architectures, we apply a paired t-test on the 5 seed-level macro F1 scores. If the Shapiro-Wilk test rejects normality ($p < 0.05$) for any pair’s score differences, we substitute the Wilcoxon signed-rank test (a non-parametric alternative that does not assume normally distributed differences). We report p-values with Bonferroni correction for multiple comparisons (10 pairwise comparisons among 5 deep learning architectures). Significance threshold: $p < 0.05$ after correction. We choose paired tests because each seed pair shares the same data split and initialization scheme, naturally pairing comparisons.

4.3.2 Experiment 2: Representational Convergence Analysis (H2, H2a)

Step 1. Representation extraction: For each trained deep learning model, we select the **best-performing seed** (highest validation macro F1 among the 5 seeds) as the representative model for CKA analysis. This is standard practice when a single model is needed for post-hoc analysis: using the best seed avoids analyzing a potentially underfit run, while the 5-seed performance statistics in Experiment 1 already capture variance. We then pass the full test set through this model. At the **penultimate layer** (immediately before the linear classification head), extract the activation vector for each test track. This produces, for each architecture, a representation matrix:

$$R_{\text{arch}} \in \mathbb{R}^{N_{\text{test}} \times D_{\text{hidden}}}$$

where $N_{\text{test}} \approx 2,505$ and D_{hidden} is the architecture’s hidden dimension (256 for all models as currently configured, though this may vary if final sizing adjusts).

Step 2. Inter-architecture CKA: For each pair of deep learning architectures (i, j) , compute linear CKA (Kornblith et al., 2019) between R_i and R_j . Formally, linear CKA is defined as:

$$\text{CKA}(R_i, R_j) = \frac{\text{HSIC}(K_i, K_j)}{\sqrt{\text{HSIC}(K_i, K_i) \cdot \text{HSIC}(K_j, K_j)}}$$

where $K_i = R_i R_i^T$ and $K_j = R_j R_j^T$ are the Gram matrices, and

where HSIC (Hilbert-Schmidt Independence Criterion) measures statistical dependence between two kernel matrices. In practice, we compute CKA using the `torch-cka` library, which implements this formula efficiently via matrix operations on GPU without requiring explicit kernel matrix construction. This produces a 5×5 CKA similarity matrix across {CNN, LSTM, Transformer, Mamba-1, Mamba-2}.

Step 3. Architecture-to-classical-features CKA: For each deep learning architecture, compute linear CKA between the penultimate-layer representations and: (a) The **full** classical feature vector (~ 518 features) for each test track, (b) **MFCCs only** (40 features), (c) **Chroma only** (24 features), and (d) **Spectral contrast only** (14 features).

This produces, for each architecture, a **feature alignment profile**: a vector [CKA_MFCC, CKA_chroma, CKA_spectral_contrast] describing which aspects of musical structure the architecture’s learned representations most resemble.

Step 4. Correlation with classification performance: Test H2’s prediction: compute Spearman rank correlation between each architecture’s CKA(learned representations, full classical features) and its test-set macro F1. We use Spearman (rank-based) rather than Pearson (linear) correlation for two reasons: (a) with only 5 data points (one per deep architecture), we cannot assume the relationship is linear or that values are normally distributed. (b) H2’s prediction is ordinal - “architectures with *higher* alignment classify *more accurately*” - which is a rank-order claim. A significant positive Spearman ρ supports H2. Computed via `scipy.stats.spearmanr`.

CKA baselines:

Untrained baseline: Compute CKA between randomly initialized (untrained) networks and classical features. This establishes the “chance-level” similarity - the floor below which no meaningful learning has occurred. This is a *sanity check*, not the convergence threshold.

Within-architecture cross-seed baseline (convergence ceiling): Compute CKA between different seeds of the same architecture (e.g., Transformer-seed-1 vs. Transformer-seed-2). This establishes the *upper bound* on representational similarity achievable by models sharing the same inductive bias but differing in random initialization. H2 is supported if inter-architecture CKA (e.g., Mamba vs. Transformer) approaches this ceiling; H2 is falsified if inter-architecture CKA is substantially lower than the within-architecture ceiling despite both exceeding the untrained floor.

CKA implementation details: The primary metric is linear CKA, computed via `torch-cka`. All CKA computations use test-set representations only. CKA naturally handles differing hidden dimensions across architectures, as it operates on the Gram matrices RR^T , which are $N_{\text{test}} \times N_{\text{test}}$ regardless of hidden dimension.

Robustness check: We additionally compute RBF CKA with σ set via the median heuristic ($\sigma = 0.5 \times \text{median pairwise Euclidean distance in the representation space}$). Linear CKA measures only second-order (linear) statistical dependence between representations. If two architectures learn representations related by a nonlinear transformation, linear CKA will underestimate their true similarity. RBF CKA uses a Gaussian kernel that captures nonlinear relationships. The median heuristic for bandwidth selection is a standard, data-adaptive choice that avoids manual tuning (Gretton et al., 2012). If linear and RBF CKA produce qualitatively similar results, our conclusions are robust to the choice of kernel.

Known limitation (pooling confound): The penultimate-layer representations compared via CKA are extracted *after* each architecture’s temporal pooling operation (global

average pooling for CNN, attention-weighted pooling for LSTM, [CLS] token for Transformer, mean pooling for Mamba). Different pooling mechanisms produce vectors with different geometric properties, which means CKA similarity captures the combined effect of the sequence model *and* its pooling strategy. This is a deliberate design choice: the pooling operation is part of each architecture’s inductive bias for summarizing a sequence, and comparing complete systems (sequence model + pooling) is the operationally relevant comparison for classification. This approach follows the precedent set by Raghu et al. (2022), who compared ViT [CLS] token representations against CNN global-average-pooled features.

Robustness check: To partially disentangle the sequence model from the pooling mechanism, we additionally compute CKA at an **intermediate layer** (layer 2 of 4) using time-averaged hidden states - i.e., averaging each architecture’s hidden representations across the time dimension at a layer where architecture-specific pooling has not yet been applied. This check is motivated by Ollershaw et al. (2022), who found that CNN, LSTM, and Transformer exhibit qualitatively different layer-wise CKA patterns (progressive hierarchy, bottom-up convergence, and irregular correlation respectively), suggesting that the choice of extraction layer interacts with architecture type. If intermediate-layer CKA and penultimate-layer CKA produce qualitatively similar convergence patterns, the conclusions are robust to both the pooling confound and the layer-selection choice.

4.3.3 Experiment 3: Interpretable Error Analysis (H3)

Prerequisite — feature group collinearity analysis: Before interpreting CKA alignment profiles per feature group, we must verify that the feature groups are sufficiently independent to support separate attribution. Compute the inter-group correlation matrix: for each pair of feature groups (MFCC, chroma, spectral contrast), calculate the canonical correlation or the mean absolute Pearson correlation between all cross-group feature pairs using the training set, reported as a 3×3 correlation matrix.

If feature groups are approximately independent (mean cross-group $|r| < 0.3$): proceed with H3 analysis as designed - each group captures a distinct musical dimension, and CKA alignment per group can be interpreted independently.

If feature groups show moderate-to-high collinearity (mean cross-group $|r| > 0.3$, particularly between MFCCs and spectral contrast): apply PCA whitening within each group before CKA computation, or restrict H3 analysis to the two most orthogonal groups (likely MFCCs vs. chroma, since timbral envelope features and pitch-class features are derived from fundamentally different signal properties). Report the collinearity finding transparently and qualify interpretations accordingly.

Step 1. Per-architecture confusion matrices: From Experiment 1, extract the 16×16 confusion matrix for each architecture on the test set.

Step 2. Identify top-confused genre pairs: For each architecture, identify the k genre pairs most frequently confused (highest off-diagonal confusion matrix entries, normalized by class size).

Step 3. Characterize genre similarity per feature group: For each genre pair, compute inter-genre distance separately in MFCC space, chroma space, and spectral contrast space (using the classical feature vectors from the training set, computed as the Euclidean distance between genre-centroid vectors via `scipy.spatial.distance`). Because each feature group *by definition* encodes a specific musical dimension - MFCCs encode timbral texture (spectral envelope shape), chroma encodes harmonic content (pitch-class energy distribution), spectral contrast encodes peak-to-valley frequency balance (see §3.3 for citations establishing these definitions) - distance in each feature space directly corresponds to difference along that musical dimension. Genres that are close in chroma space but far in MFCC space therefore share similar harmonic vocabulary but differ in timbral texture (e.g., similar chord progressions played by different instruments or production styles).

Step 4. Test H3’s prediction: For each architecture: do its most-confused genre pairs correspond to genres that are similar along the feature dimensions where that architecture has *low* CKA alignment? Specifically: - Compute Spearman correlation (via `scipy.stats.spearmanr`) between an architecture’s feature-group CKA scores and the feature-group distances of its most-confused genre pairs. - If an architecture has low CKA_chroma but high CKA_MFCC, H3 predicts it confuses genres that differ primarily in chroma (harmonic content) but are similar in MFCCs (timbre).

Step 5. Musicological interpretation: For the most salient confusion-alignment connections, provide qualitative musicological analysis grounded in the CKA results. We note CKA does not directly output statements like “this model encodes instrumentation” or “this model captures harmonic vocabulary.” What CKA provides is a *similarity score* between the architecture’s learned representations and each classical feature group. A low CKA score between an architecture and chroma features means the architecture’s internal representation has *low statistical dependence* on the pitch-class information that chroma features capture. The musicological interpretation - e.g., “Mamba confuses jazz and blues because it underrepresents harmonic differences between these genres” - is *our analytical inference* layered on top of the quantitative CKA finding, supported by the known musicological distinctions between the confused genres. We make this inference chain explicit in all qualitative claims.

Table 8: Sequence length scaling with audio duration at standard mel spectrogram resolution.

Duration	Sequence length (frames)	Calculation
30 s	$\sim 1,292$	$\lfloor 30 \times 22,050 / 512 \rfloor + 1$
60 s	$\sim 2,584$	$\lfloor 60 \times 22,050 / 512 \rfloor + 1$
120 s	$\sim 5,168$	$\lfloor 120 \times 22,050 / 512 \rfloor + 1$
300 s (5 min)	$\sim 12,919$	$\lfloor 300 \times 22,050 / 512 \rfloor + 1$

4.3.4 Experiment 4: Sequence Length Scaling

SSMs’ theoretical advantage over Transformers is specifically in long-sequence efficiency - Mamba scales linearly in sequence length ($O(N)$) while Transformer self-attention scales quadratically ($O(N^2)$). At the standard 30-second / $\sim 1,292$ -token configuration used in Experiments 1-3, this difference may be marginal. Testing at longer sequences is essential because: (1) it directly evaluates the architectural property that most strongly differentiates Mamba from Transformer - without it, we test these architectures only in the regime where their theoretical differences are smallest (2) it has direct implications for H2 and H2a - if representational convergence holds at 1,292 tokens but breaks down at 5,000+, it suggests that the shared representational subspace is an artifact of short sequences rather than a fundamental property of the task (3) it addresses practical deployment questions for full-track music analysis (e.g., automatic tagging of 3-5 minute songs), where memory constraints force Transformer-based systems to chunk audio while Mamba can process full tracks in a single forward pass.

We then extend evaluation to longer audio segments using FMA tracks beyond the standard 30-second clips at 30s, 60s, 120s, and full-track durations (where available). At each duration, we report: - Macro F1 and accuracy for all deep learning architectures - Peak GPU memory and wall-clock training/inference time - CKA alignment profiles (recomputed at each sequence length)

At standard mel spectrogram resolution (hop length 512, sample rate 22,050 Hz), the sequence lengths scale roughly as:

4.3.5 Predictions

From H1: At $\sim 1,292$ tokens, Transformer and Mamba may show comparable memory usage. The memory/speed gap should widen monotonically with sequence length. At $\sim 5,000$ -13,000 tokens, Transformer’s quadratic attention cost should dominate practical compute; if Mamba maintains accuracy while Transformer requires chunking or memory-reduction strategies (e.g., sparse attention), this provides direct empirical evidence of SSMs’ scaling advan-

tage for audio - the advantage that motivated Mamba’s design (Gu & Dao, 2024). This prediction is supported by empirical results in adjacent audio tasks: Audio Mamba matched AST (Audio Spectrogram Transformer) on AudioSet and VGGSound with linear scaling (Yadav & Tan, 2024).

From H2: Classical features (which are fixed-length summary statistics computed over the entire clip regardless of duration) may become less aligned with deep representations as sequence length increases. At longer durations, deep models can capture fine-grained temporal dynamics (e.g., structural changes across a full song: intro $\rightarrow$ verse $\rightarrow$ chorus $\rightarrow$ bridge) that fixed-window statistics cannot represent. If the convergence pattern from H2 weakens at longer sequences, it suggests representational convergence is a property of short, homogeneous clips - not of genre classification in general.

To avoid confounding sequence length with out-of-distribution data, we restrict Experiment 4 to the **untrimmed source recordings of the same tracks present in the FMA Medium test set**. FMA Medium’s 30-second clips are excerpts from longer source recordings available in the FMA archive; we access these full-length files using the same track IDs. This ensures that any performance differences across sequence lengths are attributable to the length itself - not to different musical content, recording conditions, or genre distributions. Tracks shorter than the target duration at a given condition are excluded from that condition’s evaluation. 48 GBs of VRAM should accommodate sequences up to $\sim 13,000$ tokens for all architectures without memory overflow. *Tentatively, we plan to report 30s results (Experiments 1-3); sequence length scaling results will be completed for the final paper.*

4.4 Evaluation Protocol & Metrics

This section defines and consolidates all metrics reported in the Results section. The metrics below are computed as part of the experiments described in §4.3 - they are not a separate evaluation pass. This section serves as a self-contained reference so that a reviewer can verify exactly what numbers appear in every table and figure without re-reading the experimental procedures.

Primary classification metrics (per MARBLE benchmark protocol; Yuan et al., 2023):

Per-class metrics (for Experiment 3):

Representational similarity metrics (for Experiment 2):

Efficiency metrics (for H1):

Statistical reporting. All deep learning results: mean $\pm$ std across 5 seeds. Pairwise significance tests with Bonferroni correction. Effect sizes (Cohen’s d) reported alongside p-values.

Table 9: Primary classification metrics per the MARBLE benchmark protocol.

Metric	Definition	Role
Macro F1	Unweighted mean of per-class F1 scores	Primary reported metric; treats all 16 genres equally regardless of class frequency
Accuracy	Correct predictions / total predictions	Secondary metric; reported for comparability with prior work

Table 10: Per-class metrics for interpretable error analysis (Experiment 3).

Metric	Definition
Per-class precision	TP / (TP + FP) per genre
Per-class recall	TP / (TP + FN) per genre
Full 16×16 confusion matrix	Per architecture

5. Benchmarking & Baselines

Content to be populated with experimental results.

5.1 Performance Baselines

- Random guess: $1/16 = 6.25\%$ accuracy
- Majority class: accuracy of most frequent genre (to be reported)
- Classical ML (SVM, RF, XGBoost): from our Experiment 1
- Published FMA results where comparable (noting differences in splits, preprocessing, or pretraining)

5.2 CKA Baselines

- Untrained networks: establishes chance-level representational similarity
- Within-architecture (cross-seed) CKA: establishes the similarity ceiling for trained models of the same type

6. Results

Content to be populated after experiments are completed.

6.1 Planned Deliverables

1. **Table 1:** Performance comparison - macro F1, accuracy (mean $\pm$ std) for all 8 models, with pairwise significance indicators.
2. **Table 2:** Efficiency comparison - training time, peak memory, inference throughput for all 5 deep learning models.
3. **Figure 1:** 5×5 CKA heatmap - inter-architecture representational similarity at the penultimate layer.

Table 11: Representational similarity metrics for CKA analysis (Experiment 2).

Metric	Definition	Source
Linear CKA	Normalized HSIC with linear kernel; range [0, 1]	Kornblith et al. (2019)
RBF CKA	CKA with RBF kernel; robustness check	Kornblith et al. (2019)
Feature-group CKA	CKA between learned representations and individual feature groups (MFCC, chroma, spectral contrast)	Novel application

Table 12: Efficiency metrics for performance benchmarking (H1).

Metric	Definition
Wall-clock training time	Total time to convergence or early stopping
Peak GPU memory	Maximum GPU memory allocated during training
Inference throughput	Test tracks classified per second

4. **Figure 2:** Feature-group CKA alignment profiles - bar chart or radar plot showing [CKA_MFCC, CKA_chroma, CKA_spectral_contrast] for each deep learning architecture.
5. **Figure 3:** Scatter plot - CKA(architecture, classical features) vs. macro F1 across architectures (testing H2).
6. **Figure 4:** Selected confusion matrices (e.g., Transformer vs. Mamba-2) with annotations connecting confusion patterns to alignment profiles.
7. **Table 3:** Top-5 confused genre pairs per architecture, with feature-group distances and CKA alignment scores.

7. Discussion & Future Work

Content to be developed based on experimental results.

7.1 Interpretation of Results

- **If H2 confirmed:** Implications for the “feature learning” narrative in deep audio ML. Why does architectural sophistication matter if the optimal representation was known? Hypothesis: architectures differ in sample efficiency, not representational target.
- **If H2 falsified:** Which architectures diverge, and what do they learn instead? Does divergence correlate with better or worse performance?
- **If H2a confirmed:** Structural argument linking

Mamba’s linear recurrence to IIR filters underlying MFCCs. Implication: SSM inductive bias converges toward DSP because it *is* a generalization of DSP.

7.2 Limitations

- From-scratch only; CKA captures second-order statistics only; FMA Medium only; $\sim 1,292$ tokens may not stress-test long-range architectural differences sufficiently (addressed partially by Experiment 4).

7.3 Future Work

- *Pretrained models*: Evaluate the same architectural spectrum under MARBLE’s pretraining protocol. Do pretrained representations converge more or less than from-scratch representations?
- *Non-music audio*: Extend to speech (LibriSpeech), environmental sound (ESC-50), and general audio (AudioSet) to test whether representational convergence is music-specific or a general property of audio classification.
- *Emerging architectures*: Mamba-3 (pending review at ICLR 2026), Test-Time Training layers (Sun et al., 2024), and hybrid architectures (Zamba, Jamba, Conformer) were excluded from this study to maintain clean single-mechanism comparisons. Extending the CKA analysis to hybrid models would reveal whether combining mechanisms (e.g., SSM + attention) produces representations that are superpositions of the constituent mechanisms’ profiles or something qualitatively new.
- *Advanced augmentation*: Mixup (Zhang et al., 2018) and CutMix were excluded to avoid architecture-augmentation interaction confounds. Future work could systematically study whether augmentation strategy interacts with inductive bias in the CKA alignment space.
- *Information-theoretic analysis*: Probing reconstruction loss vs. classification accuracy as a complementary lens on the information bottleneck (Tishby et al., 2000).
- *Neuroscientific validation*: Pepino et al. (2025) showed via RSA that audio model performance correlates with alignment to human auditory cortex (fMRI). Extending our CKA analysis to include brain-derived representational dissimilarity matrices would test whether from-scratch architectures that converge toward classical features also converge toward biological auditory representations.
- *Theoretical framing*: If H2 is confirmed, the finding would be consistent with the Platonic Representation Hypothesis (Huh et al., 2024), which predicts that sufficiently capable models converge toward shared representational structure. Our single-task, single-dataset study cannot establish that claim in general, but could

provide domain-specific evidence.

8. Conclusion

Content to be added. Estimated scope: the study’s contributions, the primary finding, the interpretability contribution, and the broader implication for understanding architectural inductive bias in sequential classification.

9. References

- [1] G. Tzanetakis and P. Cook. Musical genre classification of audio signals. *IEEE Trans. Speech and Audio Processing*, 10(5):293-302, 2002. <https://www.cs.cmu.edu/~{ }gtzan/work/pubs/tsap02gtzan.pdf>
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017. <https://arxiv.org/abs/1706.03762>
- [3] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *COLM*, 2024. <https://arxiv.org/abs/2312.00752>
- [4] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *ICML*, 2024. <https://arxiv.org/abs/2405.21060>
- [5] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *ICML*, 2019. <https://arxiv.org/abs/1905.00414>
- [6] R. Yuan, Y. Ma, Y. Lin, Y. Tian, et al. MARBLE: Music audio representation benchmark for universal evaluation. In *NeurIPS Datasets and Benchmarks*, 2023. <https://arxiv.org/abs/2306.10548>
- [7] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proc. Interspeech*, pages 2613-2617, 2019. <https://arxiv.org/abs/1904.08779>
- [8] A. Mishra and N. Akhtar. Music genre classification using machine learning techniques. arXiv:2509.01762, 2025. <https://arxiv.org/abs/2509.01762>
- [9] S. Chatterjee et al. Audio processing using pattern recognition for music genre classification. arXiv:2410.14990, 2024. <https://arxiv.org/abs/2410.14990>
- [10] C. Liu et al. Bottom-up broadcast neural network for music genre classification. arXiv:1901.08928, 2019. <https://arxiv.org/abs/1901.08928>
- [11] Y. Zhang and T. Li. Music genre classification with parallel convolutional neural networks and capuchin search algorithm. *Scientific Reports*, 15(1):9580, 2025. <https://doi.org/10.1038/s41598-025-90619-7>

- [12] L. Feng, S. Liu, and J. Yao. Music genre classification with paralleling recurrent convolutional neural network. arXiv:1712.08370, 2017. <https://arxiv.org/abs/1712.08370>
- [13] C. P. Tang, K. L. Chui, Y. K. Yu, Z. Zeng, and K. H. Wong. Music genre classification using a hierarchical long short term memory (LSTM) model. In *Proc. SPIE*, volume 10828, page 108281B, 2018. <https://doi.org/10.1117/12.2501763>
- [14] X. Luo. Automatic music genre classification based on CNN and LSTM. In *Proc. CMLAI*, volume 39, 2023. <https://doi.org/10.54097/hset.v39i.6494>
- [15] T. Chu Ba, T. D. T. Le, and L. T. Van. Music genre classification using deep neural networks and data augmentation. *Entertainment Computing*, 53:100929, 2025. <https://doi.org/10.1016/j.entcom.2025.100929>
- [16] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *ICLR*, 2022. <https://arxiv.org/abs/2111.00396>
- [17] D. Liu, M. Gumusayak, D. Gordon, and B. Kulis. Mo-Mamba: A lightweight music oriented Mamba-based model for music information retrieval tasks. OpenReview, 2026 (withdrawn). <https://openreview.net/forum?id=TUBfsCpciR>
- [18] S. Yadav and Z.-H. Tan. Audio Mamba: Selective state spaces for self-supervised audio representations. In *Inter-speech*, 2024. <https://arxiv.org/abs/2406.02178>
- [19] S. Shams, S. S. Dindar, X. Jiang, and N. Mesgarani. SSAMBA: Self-supervised audio representation learning with Mamba state space model. In *IEEE Spoken Language Technology Workshop (SLT)*, pages 1053-1059, 2024. <https://doi.org/10.1109/SLT61566.2024.10832304>
- [20] D. Mu, Z. Zhang, H. Yue, Z. Wang, J. Tang, and J. Yin. SELD-Mamba: Selective state-space model for sound event localization and detection with source distance estimation. arXiv:2408.05057, 2024. <https://arxiv.org/abs/2408.05057>
- [21] A. Bairwa, S. Bhat, T. Sawant, and M. Bohra. StructMamba: Structured harmonic and temporal music analysis via dual-axis Mamba and attention. *IEEE Access*, pages 1-1, 2025. <https://doi.org/10.1109/ACCESS.2025.3617184>
- [22] A. Fratticioli. MambaTransfer: Raw audio timbre transfer with Mamba. MSc thesis, Politecnico di Milano, 2025. https://www.politesi.polimi.it/retrieve/5730b355-959a-410b-8ec4-aab92f4723e9/2025_04_Fratticioli_Executive_Summary.pdf
- [23] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? arXiv:2108.08810, 2022. <https://arxiv.org/abs/2108.08810>
- [24] T. Nguyen, M. Raghu, and S. Kornblith. Do wide and deep networks learn the same things? Uncovering how neural network representations vary with width and depth. In *ICLR*, 2021. <https://arxiv.org/abs/2010.15327>
- [25] A. Ollerenshaw, M. A. Jalal, and T. Hain. Insights on neural representations for end-to-end speech recognition. arXiv:2205.09456, 2022. <https://arxiv.org/abs/2205.09456>
- [26] A. Pasad, J.-C. Chou, and K. Livescu. Layer-wise analysis of a self-supervised speech representation model. In *ASRU*, 2021. <https://arxiv.org/abs/2107.04734>
- [27] A. Pasad et al. What do self-supervised speech models know about words? arXiv:2307.00162, 2023. <https://arxiv.org/abs/2307.00162>
- [28] M. Lou, Y. Fu, and Y. Yu. SparX: A sparse cross-layer connection mechanism for hierarchical vision Mamba and Transformer networks. In *Proc. AAAI*, 39(18):19104-19114, 2025. <https://doi.org/10.1609/aaai.v39i18.34103>
- [29] N. M. Hoang, D. X. Long, C.-D. Nguyen, M.-Y. Kan, and L. A. Tuan. A comparative analysis of contextual representation flow in state-space and Transformer architectures. arXiv:2510.06640, 2026. <https://arxiv.org/abs/2510.06640>
- [30] C. Plachouras, E. Benetos, and J. Pauwels. Learning music audio representations with limited data. In *ICASSP*, pages 1-5, 2025. <https://doi.org/10.1109/ICASSP49660.2025.10887766>
- [31] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson. FMA: A dataset for music analysis. arXiv:1612.01840, 2017. <https://arxiv.org/abs/1612.01840>
- [32] B. McFee, C. Raffel, D. Liang, D. Ellis, M. McVicar, E. Battenberg, and O. Nieto. librosa: Audio and music signal analysis in Python. In *Proc. SciPy*, pages 18-24, 2015. <https://doi.org/10.25080/Majora-7b98e3ed-003>
- [33] K. Choi, G. Fazekas, M. Sandler, and K. Cho. Convolutional recurrent neural networks for music classification. In *ICASSP*, pages 2392-2396, 2017. <https://doi.org/10.1109/ICASSP.2017.7952585>
- [34] B. Logan. Mel frequency cepstral coefficients for music modeling. In *Proc. 1st Int. Symposium Music Information Retrieval*, 2000.
- [35] M. Bartsch and G. Wakefield. Audio thumbnailing of popular music using chroma-based representations. *IEEE Trans. Multimedia*, 7(1):96-104, 2005. <https://doi.org/10.1109/TMM.2004.840597>
- [36] D.-N. Jiang, L. Lu, H.-J. Zhang, J.-H. Tao, and L.-H. Cai. Music type classification by spectral contrast feature. In *Proc. IEEE ICME*, volume 1, 2002. <https://doi.org/10.1109/ICME.2002.1035731>
- [37] A. Flexer and D. Schnitzer. Effects of album and artist filters in audio similarity computed for very large music databases. *Computer Music Journal*, 34(3):20-28, 2010. https://www.mitpressjournals.org/doi/pdf/10.1162/COMJ_a.00004
- [38] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *ICLR*, 2018. <https://arxiv.org/abs/1710.09412>

- [39] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. <https://arxiv.org/abs/1711.05101>
- [40] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. arXiv:1708.02002, 2018. <https://arxiv.org/abs/1708.02002>
- [41] Y. Sun et al. Learning to (learn at test time): RNNs with expressive hidden states. arXiv:2407.04620, 2024. <https://arxiv.org/abs/2407.04620>
- [42] J. Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1-30, 2006. <https://dl.acm.org/doi/10.5555/1248547.1248548>
- [43] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723-773, 2012. <https://dl.acm.org/doi/10.5555/2188385.2188410>
- [44] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. arXiv:physics/0004057, 2000. <https://arxiv.org/abs/physics/0004057>
- [45] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv:2307.08691, 2023. <https://arxiv.org/abs/2307.08691>
- [46] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced Transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. <https://doi.org/10.1016/j.neucom.2023.127063>
- [47] M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? arXiv:2108.08810, 2022. <https://arxiv.org/abs/2108.08810>
- [48] L. Pepino, P. Riera, J. Kamienkowski, and L. Ferrer. Better audio representations are more brain-like: Linking model-brain alignment with performance in downstream auditory tasks. arXiv:2511.16849, 2025. <https://arxiv.org/abs/2511.16849>
- [49] M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. arXiv:2405.07987, 2024. <https://arxiv.org/abs/2405.07987>

10. Appendix

Table 13: Abbreviations used in this paper, listed in order of first appearance.

Abbreviation	Full Form
MIR	Music Information Retrieval
STFT	Short-Time Fourier Transform
ML	Machine Learning
SVM	Support Vector Machine
XGBoost	eXtreme Gradient Boosting
MFCCs	Mel-Frequency Cepstral Coefficients
DSP	Digital Signal Processing
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
SSM	(Selective/Structured) State Space Model
MARBLE	Music Audio Representation Benchmark for Universal Evaluation
NLP	Natural Language Processing
CKA	Centered Kernel Alignment
GPU	Graphics Processing Unit
IIR	Infinite Impulse Response
DCT	Discrete Cosine Transform
FMA	Free Music Archive
GRU	Gated Recurrent Unit
HiPPO	High-order Polynomial Projection Operators
RF	Random Forest
SELD	Sound Event Localization and Detection
CCA	Canonical Correlation Analysis
SVCCA	Singular Vector Canonical Correlation Analysis
ViT	Vision Transformer
RBF	Radial Basis Function
ASR	Automatic Speech Recognition
RNN	Recurrent Neural Network
CQT	Constant-Q Transform
PCEN	Per-Channel Energy Normalization
ReLU	Rectified Linear Unit
FFN	Feed-Forward Network
RoPE	Rotary Position Embeddings
AST	Audio Spectrogram Transformer
HuBERT	Hidden-Unit BERT for Audio
TCN	Temporal Convolutional Network
MERT	Music Enhanced Representation Transformer
AdamW	Adam with Decoupled Weight Decay
LR	Learning Rate
VRAM	Video Random Access Memory
CUDA	Compute Unified Device Architecture
HSIC	Hilbert-Schmidt Independence Criterion
PCA	Principal Component Analysis
RSA	Representational Similarity Analysis
fMRI	functional Magnetic Resonance Imaging